Laboratorio 3 Servicio patrón stateful en AWS

Adaptación del ejercicio de Compute Engine a Amazon EC2 y EBS

Katherine Aguirre

Infraestructura Tecnológica para Inteligencia Artificial

Diplomatura en Desarrollo de Soluciones de Inteligencia Artificial Generativa en la Nube

Universidad Nacional de Luján

13 de octubre de 2026

Resultado

El laboratorio se implementó en AWS con infraestructura como código. La instancia EC2 ejecutó el servicio patrón y Ollama, mientras que un volumen EBS independiente conservó los datos y los pesos. Las comprobaciones se realizaron sobre recursos reales en us-east-1; los identificadores de cuenta, instancia, volumen e IP pública fueron omitidos de esta entrega.

Equivalencias usadas

Guía original	Implementación en AWS
Compute Engine	Amazon EC2 c7i-flex.large
Persistent Disk	Amazon EBS gp3 cifrado de 10 GiB
Regla de firewall	Security group TCP 8080 restringido a /32
SSH	AWS Systems Manager Session Manager

Adaptación de la arquitectura de GCP a AWS

La guía original propone construir el laboratorio manualmente con gcloud para comprender cada pieza. En AWS se mantuvieron los mismos objetivos, pero la infraestructura se expresó con Terraform para que la creación, el reemplazo y la limpieza fueran repetibles y verificables.

Capa	Guía original	Implementación realizada
Cómputo	Compute Engine e2-small/e2-medium	EC2 c7i-flex.large con Amazon Linux 2023
Datos	Persistent Disk separado	EBS gp3 cifrado de 10 GiB y ciclo de vida independiente
Acceso	gcloud compute ssh	Systems Manager con rol y perfil IAM; sin publicar SSH
Red	Regla de firewall por tag	Security group dedicado, TCP 8080 limitado a una IPv4 /32
Arranque	startup-script en metadata	user_data de EC2 generado desde user-data.sh.tftpl
Automatización	Comandos gcloud manuales	Plan guardado de Terraform, pruebas y aplicación controlada

Cómo se llegó a la solución

Primero se identificó la intención de cada comando de GCP: crear cómputo, separar el almacenamiento, limitar la red, instalar el runtime y demostrar persistencia. Después se modeló cada intención con un recurso equivalente de AWS. El entorno quedó en terraform/aws/envs/lab3, el módulo reutilizable en terraform/aws/modules/ec2-stateful y las validaciones en tests/lab3.tftest.hcl. Los ejemplos originales de GCP no fueron modificados.

Decisión arquitectónica para Terraform en AWS

Terraform no era un requisito del Lab 3, cuya guía propone ejecutar comandos gcloud manualmente para comprender cada recurso. Sin embargo, el repositorio template de la cátedra ya incluía infraestructura como código con OpenTofu o Terraform para GCP a partir de la Clase 2. Por esa razón se reutilizó la misma convención al trasladar el ejercicio a AWS, en lugar de introducir un mecanismo de aprovisionamiento ajeno al repositorio.

La implementación de AWS se agregó bajo terraform/aws como un árbol paralelo, sin modificar los módulos originales de GCP. Esta decisión permitió revisar los cambios antes de aplicarlos, gestionar dependencias, repetir el despliegue y efectuar una limpieza controlada. Terraform administró los recursos de nube; user_data y las pruebas funcionales verificaron la configuración y los servicios dentro de la instancia.

Parte 1 Equivalencias para VM y disco persistente

Estas equivalencias cubren las tres secciones de aprovisionamiento de la guía. Un solo plan de Terraform creó los recursos, mientras que user_data completó la preparación del sistema operativo.

Sección	Guía GCP y equivalente ejecutado en AWS	Criterio de adaptación
1.1	`GCP: gcloud compute instances create ... --metadata=startup-script` `AWS: terraform plan -out=tfplan; terraform apply tfplan`	aws_instance creó la EC2; user_data instaló Docker, Git y utilidades sobre Amazon Linux 2023.
1.2	`GCP: gcloud compute disks create; gcloud compute instances attach-disk` `AWS: aws_ebs_volume + aws_volume_attachment dentro del mismo plan`	El EBS quedó separado del disco raíz y no se reemplazó junto con la instancia.
1.3	`GCP: gcloud compute ssh; mkfs.ext4; blkid; mount; editar /etc/fstab` `AWS: user_data ejecutó mkfs.ext4, obtuvo el UUID y escribió fstab; findmnt verificó el montaje`	Se conservó el montaje por UUID, pero se automatizó y se añadieron controles para no formatear un disco con datos.

Parte 2 Equivalencias para el servicio y la persistencia

El servicio patrón conservó el comportamiento de la guía. Los comandos Linux y Docker siguieron siendo válidos; cambiaron el acceso a la VM, la apertura de red y la forma de reemplazar la instancia.

Sección	Guía GCP y equivalente ejecutado en AWS	Criterio de adaptación
2.1	GCP: crear manualmente el servidor con /, /healthz y /metrics AWS: user_data clonó el repositorio y utilizó app/ con los mismos endpoints	Se reutilizó el servicio patrón del template para concentrar la adaptación en la infraestructura.
2.2	GCP: docker build; docker run -p 8080:8080 -v <disco>:/data AWS: docker build -t app:lab3 app; docker run -v /mnt/lab3-data/app:/data	El bind mount mantuvo el contador fuera de la capa descartable del contenedor.
2.3	GCP: gcloud compute firewall-rules create; curl http://<IP>:8080 AWS: aws_vpc_security_group_ingress_rule; curl desde CloudShell	Se publicó sólo TCP 8080 y únicamente para la IPv4 /32 de origen; SSH no quedó expuesto.
2.4	GCP: curl; docker rm -f; docker run; curl AWS: los mismos comandos se ejecutaron por SSM sobre la EC2	Docker es independiente del proveedor. El valor persistente pasó de 6 a 7 al recrear el contenedor.
2.5	GCP: instances delete; disks describe; instances create --disk AWS: terraform plan -replace=module.lab3.aws_instance.this; terraform apply <plan>	El plan mostró el EBS como no-op. La EC2 nueva heredó 7 y la primera consulta produjo 8.

Parte 3 Equivalencias para la carga de inteligencia artificial

La guía permite elegir el servidor de inferencia. Se eligió Ollama con llama3.2:1b porque ofrecía una carga reproducible en CPU y permitía almacenar los pesos en el mismo EBS persistente.

Sección	Guía GCP y equivalente ejecutado en AWS	Criterio de adaptación
3.1	GCP: elegir Ollama o un servidor propio con Transformers AWS: ollama/ollama:0.40.3 con llama3.2:1b	La imagen y el modelo se fijaron explícitamente para repetir la prueba.
3.2	GCP: docker run -v /mnt/disks/datos/ollama:/root/.ollama; ollama pull AWS: docker run -v /mnt/lab3-data/ollama:/root/.ollama; docker exec ollama ollama pull	Los pesos quedaron en EBS y sobrevivieron al reemplazo de la EC2.
3.3	GCP: diez curl a localhost:11434/api/generate con time_total AWS: el mismo benchmark se lanzó mediante SSM dentro de EC2	El endpoint de Ollama permaneció en loopback; se midieron mínimo, máximo y promedio reales.
3.4	GCP: analizar CPU frente a GPU según modelo, latencia y carga AWS: analizar c7i-flex.large, 2 vCPU, 4 GiB y diez solicitudes	CPU fue suficiente para el modelo 1B y una demo; GPU se reserva para modelos mayores o concurrencia.

Parte 4 Equivalencia para la limpieza

La limpieza mantuvo la intención de la guía: retirar todos los recursos del laboratorio y demostrar que no quedó infraestructura facturable propia del ejercicio.

Sección	Guía GCP y equivalente ejecutado en AWS	Criterio de adaptación
4.1	`GCP: instances delete; disks delete; firewall-rules delete; list` `AWS: terraform plan -destroy -out=destroy.plan; terraform apply destroy.plan; AWS CLI describe/get`	Se respaldó el estado, se aplicó el plan guardado y se verificaron Terraform vacío y recursos inexistentes.

Resultado de la adaptación

La infraestructura se definió mediante un modelo declarativo de Terraform, especificando el estado final esperado de la instancia EC2, el volumen EBS y las reglas de red. En lugar de ejecutar manualmente una secuencia de comandos para crear, asociar y configurar cada recurso, la configuración describe cómo debe quedar la infraestructura.

Terraform comparó esa configuración con los recursos existentes en AWS, calculó las acciones necesarias y gestionó sus dependencias. El plan previo permitió revisar qué recursos se crearían, modificarían o eliminarían antes de aplicar los cambios.

Sin embargo, la declaración de infraestructura no garantiza por sí sola el funcionamiento de los servicios instalados dentro de la VM. Por ello, el despliegue se complementó con verificaciones funcionales de montaje, conectividad, persistencia e inferencia. En esta adaptación, SSM sustituyó el acceso por SSH y el volumen EBS se mantuvo con un ciclo de vida independiente de la instancia EC2.

Entregable 1.1 Montaje persistente por UUID

Se verificó el volumen EBS separado del disco raíz, formateado como ext4 y montado en /mnt/lab3-data. La entrada de fstab usa el UUID estable del filesystem.

Comprobación	Resultado verificado
Dispositivo de datos	nvme1n1 · ext4 · 10 GiB
Punto de montaje	/mnt/lab3-data
UUID	01d17fc9-36d0-4cc4-b473-1f1a336a58c3
fstab	UUID=01d17fc9-36d0-4cc4-b473-1f1a336a58c3 /mnt/lab3-data ext4 defaults,x-systemd.device-timeout=10min 0 2
Resultado	Montado en lectura y escritura; 8 GiB disponibles al verificar

Interpretación

El nombre NVMe puede cambiar entre instancias, pero el UUID identifica el filesystem. La opción de timeout permite que systemd gestione la espera del dispositivo sin vincular el montaje a un nombre volátil.

Entregable 2.1 Servicio y métricas desde fuera de EC2

Las consultas se ejecutaron desde AWS CloudShell, fuera de la instancia. El security group expuso únicamente TCP 8080 al origen /32 autorizado.

Comprobación	Resultado verificado
GET /	Servicio patrón activo. Visitas persistidas en disco: 2
GET /metrics	servicio_patron_hits_total 2.0
GET /healthz	{"status":"ok"}
Exposición	TCP 8080 limitado a la IPv4 /32 de CloudShell

Resultado

El valor del contador y la métrica Prometheus coincidieron. Esto prueba conectividad externa, funcionamiento HTTP y actualización de la métrica de negocio.

Entregable 2.2 Persistencia al recrear el contenedor

Se incrementó el contador, se eliminó completamente el contenedor servicio-patron y se creó otro desde la misma imagen con el mismo bind mount de EBS.

Comprobación	Resultado verificado
Valor antes de docker rm	6
Acción	docker rm -f servicio-patron y docker run con /mnt/lab3-data/app:/data
Valor después de recrear	7
Archivo persistente	6 inmediatamente después de recrear; 7 después de la primera consulta
Conclusión	El contador continuó la secuencia y el contenedor quedó activo con el mismo bind mount.

Entregable 2.3 Persistencia al reemplazar la instancia

Terraform reemplazó EC2 y su asociación, pero conservó el volumen EBS como recurso no modificado. El bootstrap de la instancia nueva montó el mismo filesystem.

Comprobación	Resultado verificado
Plan aplicado	2 recursos agregados, 0 modificados y 2 destruidos
Volumen EBS	no-op y el mismo identificador antes y después
Plan SHA-256	59580ab1bf47ed513e9c0827beada1ea 30f285da230b317ca8659f028c439d04
Contador heredado	7 en la EC2 nueva, antes de consultar el servicio
Primera consulta en EC2 nueva	8 en la primera consulta
Resultado	El mismo EBS conservó el contador y el modelo; Terraform quedó sin cambios.

Se utilizó un plan guardado y validado por hash. La indisponibilidad se limitó al reemplazo de la instancia y al tiempo de bootstrap.

Entregable 3.1 Pesos del modelo en almacenamiento persistente

La carga de IA elegida fue Ollama 0.40.3 con llama3.2:1b. El directorio de modelos del contenedor se vinculó a /mnt/lab3-data/ollama.

Comprobación	Resultado verificado
Runtime	Ollama 0.40.3 en Docker
Modelo	llama3.2:1b
Bind mount	/mnt/lab3-data/ollama → /root/.ollama
Uso medido	1.3 GiB en /mnt/lab3-data/ollama
Acceso	API sólo en 127.0.0.1:11434 dentro de EC2

Resultado

Los pesos quedaron fuera de la capa efímera del contenedor. Recrear Ollama no exige volver a descargar el modelo mientras el EBS se conserve.

Entregable 3.2 Latencia de inferencia en CPU

Se enviaron diez solicitudes consecutivas a /api/generate con stream=false y el mismo prompt. La instancia no tenía GPU.

Solicitud	Latencia total
1	12.449 s
2	4.022 s
3	4.325 s
4	5.606 s
5	8.403 s
6	6.981 s
7	4.094 s
8	6.462 s
9	4.113 s
10	5.737 s

Máquina	Mínimo	Máximo	Promedio
c7i-flex.large · 2 vCPU · 4 GiB	4.022 s	12.449 s	6.219 s

La primera solicitud fue la más lenta por el calentamiento inicial. El promedio incluye ese costo y representa mejor una sesión corta de laboratorio.

Entregable 3.3 Decisión CPU frente a GPU

Para esta carga de laboratorio, CPU es la opción proporcionada. La medición real no justifica una GPU para uso esporádico, aunque la conclusión cambia con modelos y demanda mayores.

Criterio	Situación medida	Decisión
Modelo	llama3.2:1b cuantizado	CPU
Demanda	Diez solicitudes secuenciales de prueba	CPU
Latencia	Promedio 6.219 s; máximo 12.449 s	Aceptable para demo
Paralelismo	Sin carga concurrente sostenida	CPU
Costo	Instancia general sin acelerador	CPU
Escenario futuro	Modelo ≥7B, streaming fluido, concurrencia o SLA estricto	Evaluar GPU

Conclusión

La GPU no se justifica para llama3.2:1b y una demo académica de bajo volumen. Sí convendría evaluarla si el proyecto exige un modelo mayor, respuestas de baja latencia, múltiples usuarios simultáneos o procesamiento por lotes que mantenga el acelerador ocupado. La decisión debe repetirse con un benchmark de esa carga futura y con el costo horario vigente.

Entregable 4.1 Limpieza de los recursos

La limpieza se ejecutó con un plan de destrucción de Terraform después de guardar el estado y las evidencias. La verificación posterior confirmó que los recursos propios del laboratorio ya no permanecen en la cuenta.

Comprobación	Resultado verificado
Plan de destrucción	0 agregados, 0 modificados y 9 destruidos
Instancia EC2 del laboratorio	Terminada
Volumen EBS del laboratorio	Eliminado; AWS respondió InvalidVolume.NotFound
Security group del laboratorio	Eliminado; AWS respondió InvalidGroup.NotFound
Perfil y rol IAM	Rol y perfil eliminados; AWS respondió NoSuchEntity
Terraform	Estado con 0 recursos y plan de destrucción sin cambios

Cierre

El respaldo privado final del estado previo a la limpieza quedó identificado por el SHA-256 92E56CB0AE56BF7FBAD1EBFF796402288956E23893FA180FF8CCEDAC043FE5E8. No se incluyeron credenciales, identificadores de cuenta, IDs de recursos ni direcciones IP en este documento.